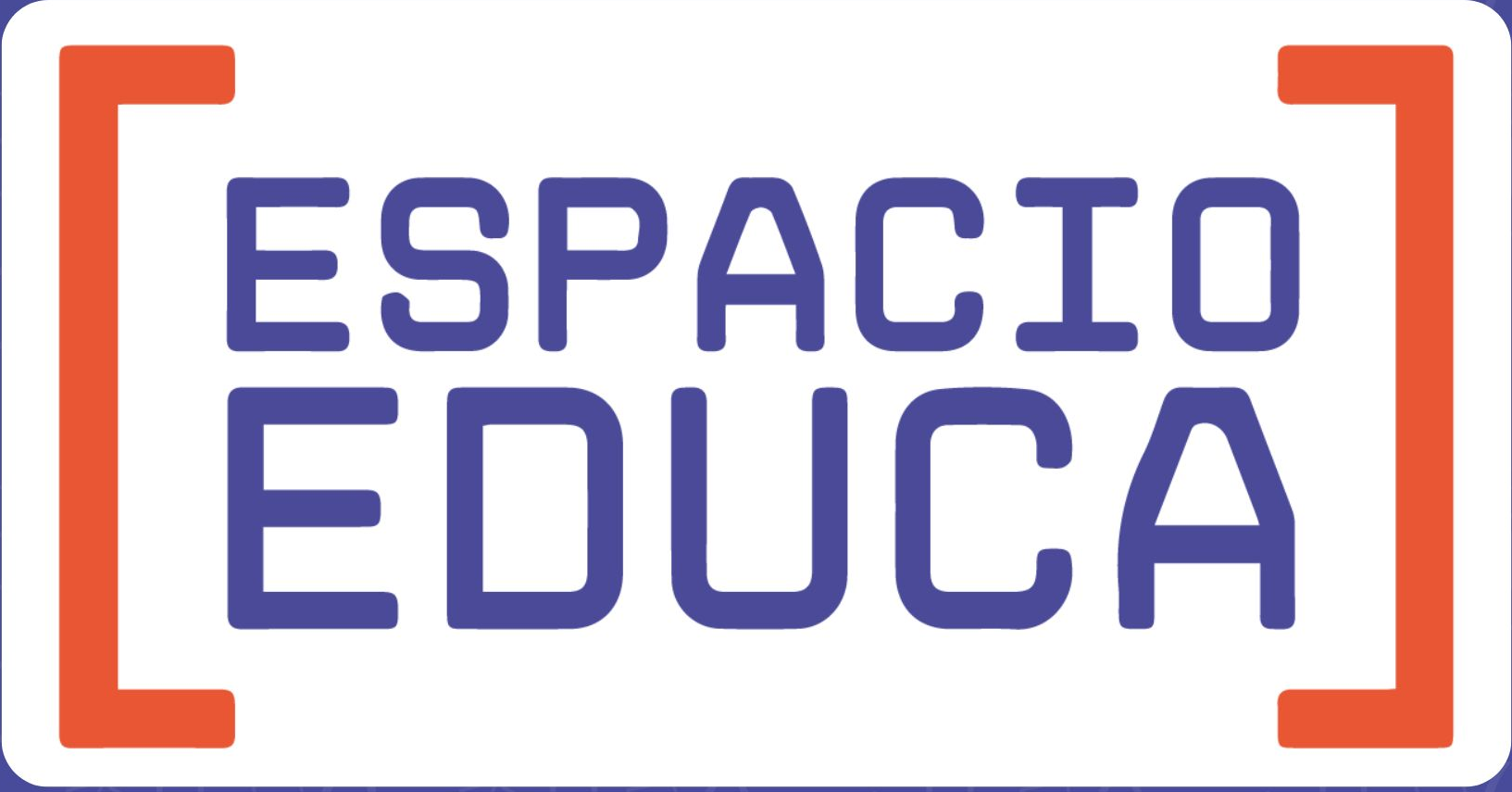




**ESPACIO
EDUCA**

[HOY VAMOS A APRENDER A...]

Crear y utilizar funciones.

Vocabulario: funciones, atributo, parámetros



Qué es una función?

Una función es un **bloque de código** diseñado para realizar una tarea específica.

Nos permite reutilizar código, mantener el orden en el programa y mejorar la claridad del código.

Cómo es reutilizable?

Cómo mantiene el orden?

Cómo hace su entendimiento más fácil?



CALSE DIRECTA



La estructura de una función básica en **JavaScript** es la siguiente:

- Palabra reservada "function"
- Nombre de la función
- Parámetros (opcionales)
- Lógica
- Retorno

```
1 function 2 sum 3 (a, b) {  
4   const c = a + b  
5   return c  
}
```

- 1 PALABRA RESERVADA
- 2 NOMBRE DE LA FUNCIÓN
- 3 PARÁMETROS
- 4 LÓGICA
- 5 RETORNO DE LA FUNCIÓN





Veamos los conceptos

¿Qué son los Parámetros?

Los parámetros son variables que usamos en la función para recibir valores.

¿Qué son los Argumentos?

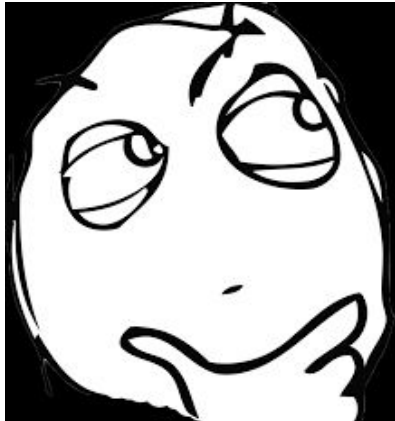
Los argumentos son los valores específicos que pasamos cuando llamamos a la función.



PREGUNTA FLASH



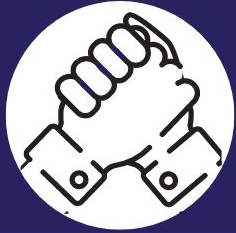
Identifiquemos cada uno de los elementos de esta función



JavaScript

```
1 function sumar(a, b) {  
2   return a + b;  
3 }  
4 console.log(sumar(3, 5));  
5
```

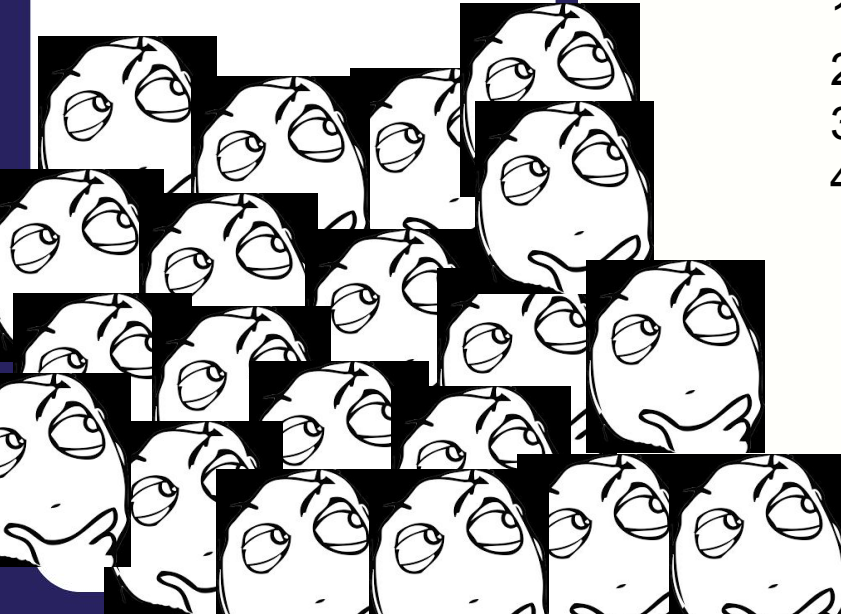




Ahora, hagámoslo
juntos

Utilicemos el ejemplo anterior y creemos una
función para **multiplicar 2 números**.

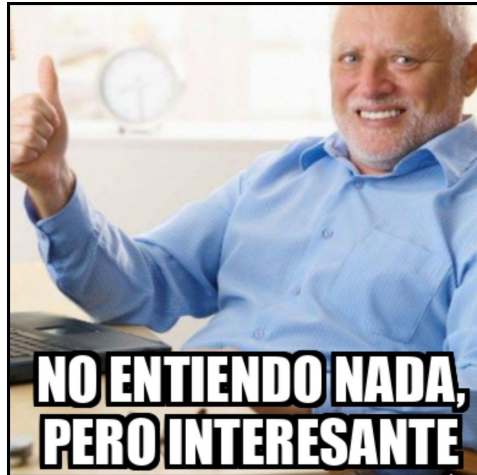
- 1- Qué nombre tendrá la función?
- 2- Qué parámetros necesitan?
- 3-Cuál es la lógica de la función?
- 4-Cuál es el resultado?



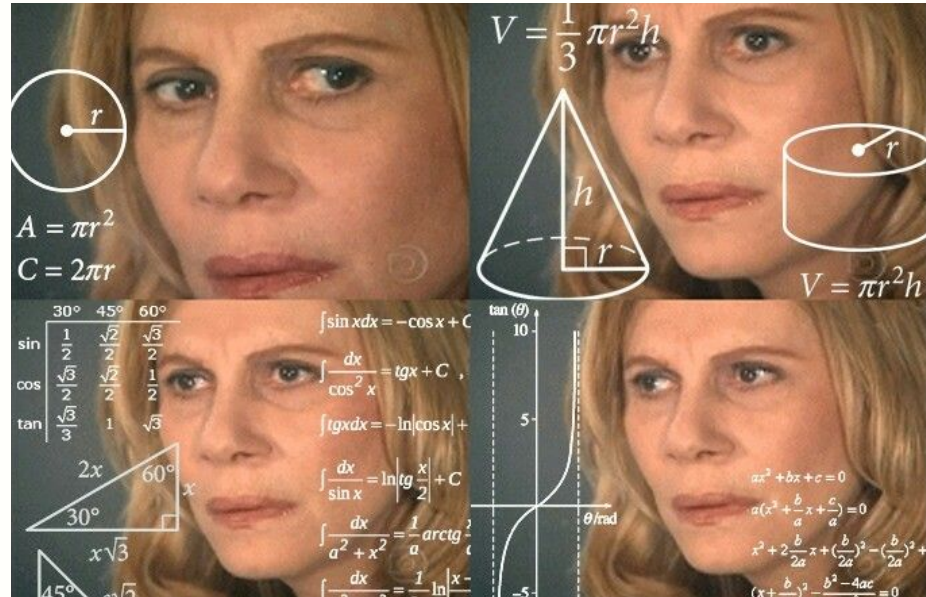
PREGUNTA FLASH



¿Cómo utilizamos
las funciones ?



Es cosa de magia...



CALSE DIRECTA



1. Definir la Función

1. Llamar a la Función

1. Funciones con Parámetros

1. Funciones con Retorno

JavaScript

```
1 function celsiusAFahrenheit(celsius) {  
2   return (celsius * 9/5) + 32;  
3 }  
4 console.log(celsiusAFahrenheit(30));
```



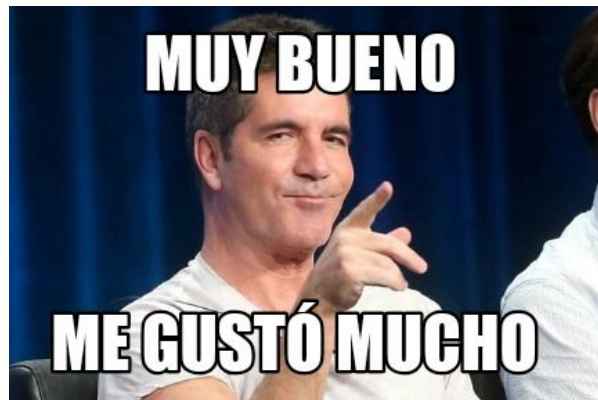


Reutilización de código

Cuando definimos una función, estamos creando una **plantilla** que realiza una tarea específica.

En lugar de escribir el mismo código varias veces, solo **llamamos a la función cada vez que necesitamos** esa tarea.

Esto es especialmente útil en aplicaciones grandes, donde ciertas acciones, como cálculos o validaciones, **se repiten**.





Facilidad de mantenimiento

¿Por qué las funciones facilitan el mantenimiento?

- 1. Localización de errores.
- 1. Actualización y mejoras.
- 1. Reutilización y pruebas aisladas.
- 1. Legibilidad y organización.





Organización del programa

¿Cómo las funciones ayudan en la organización del programa?

- 1. Modularidad
- 1. Legibilidad
- 1. Separación de responsabilidades
- 1. Escalabilidad





Funciones Anónimas

Una función anónima no tiene nombre. A menudo, se asigna a una variable para llamarla luego.

Otras formas de utilizar funciones...



JavaScript

```
1 • const saludar = function() {  
2   console.log("Hola desde una función anónima");  
3 };  
4 saludar();
```



Funciones Flecha (Arrow Functions)

Las funciones flecha (`=>`) son otra forma de escribir funciones en ES6*, que hace el código más conciso.

Otras formas de utilizar funciones...



JavaScript

```
1 const sumar = (a, b) => a + b;  
2 console.log(sumar(2, 3)); // Imprime 5
```

ES6* (ECMAScript 6) es la sexta versión del estándar ECMAScript, que agregó mejoras importantes a JavaScript.



HAZLO AHORA



Ahora, a programar.

Generador de
Contraseñas

Escribe una función que genera contraseñas aleatorias según ciertos criterios:

1. Longitud de la contraseña.
2. Incluir o no mayúsculas, números y caracteres especiales.
3. Cada criterio debe ser una opción que se pase como parámetro a la función, para que el usuario personalice su contraseña.



HAZLO AHORA



Ahora, a programar.

Generador de
Contraseñas

Entrada:

```
generarContraseña(12, { mayusculas: true,  
numeros: true, caracteresEspeciales: true })
```

Salida:

```
"gT!9aQ&7bK!p"
```



CHALLENGE ACCEPTED!

